RL-Based Neural Network Pruning for Causal Time Series Models

Aayush BISHT[1]

CentraleSupélec, Université Paris-Saclay, 91190 Gif-sur-Yvette, Paris
aayush.bisht@student-cs.fr

Abstract. Temporal Convolutional Networks (TCNs) excel at sequence modeling but carry heavy computational costs. While standard pruning methods reduce this burden, they often blindly destroy the network's learned causal mechanisms. This report formulates layer-wise network pruning as a Markov Decision Process (MDP) to automate the discovery of compression policies. We deploy a Deep Q-Network (DQN) agent to dynamically determine optimal layer-specific pruning ratios. Evaluated on a synthetic causal dataset, the DQN agent achieved a 1.63x compression ratio, removing 38.7% of total parameters while maintaining interventional causal robustness, outperforming random and uniform baselines.

Keywords: Neural Network Pruning · Reinforcement Learning · Deep Q-Network · Causal Inference.

1 Introduction

Deploying complex time series forecasting models, such as Temporal Convolutional Networks (TCNs) [2], in resource-constrained environments requires significant model compression. Neural network pruning is a widely studied approach to this problem, with methods ranging from magnitude-based weight removal [5] to structured pruning of entire filters [6]. However, standard pruning methods focus purely on observational accuracy, which risks severing the underlying causal pathways learned by the model.

The Lottery Ticket Hypothesis [4] demonstrated that sparse subnetworks can match dense performance, but identifying these subnetworks remains challenging. He et al. [1] showed that reinforcement learning can automate the selection of layer-wise compression ratios, treating pruning as a sequential decision problem. We build on this insight but extend it to the causal setting: our agent must preserve not only predictive accuracy but also the model's robustness under interventional distribution shifts.

We formulate layer-wise pruning as a Markov Decision Process (MDP) and deploy a Deep Q-Network (DQN) agent to determine optimal layer-specific pruning ratios. Evaluated on the Causal Chamber dataset [3], the agent achieves a 1.63x compression ratio while maintaining interventional causal robustness, outperforming random and uniform baselines.

2 Background / Environment Setup

The pruning process is formulated as an MDP where the RL agent sequentially prunes layers of a pre-trained TCN base model.

- **State Space:** The state provides the agent with a 7-dimensional vector: (1) normalized layer index i/L, (2) layer parameter count normalized by total parameters, (3) current layer density (fraction of non-zero weights), (4) cumulative network sparsity, (5) ratio of current validation loss to baseline loss, (6–7) one-hot encoding of the layer type (Conv1d or Linear).
- **Action Space:** The action space is discrete. For each layer, the agent selects one of 5 distinct pruning ratios: 10%, 20%, 30%, 40%, or 50%. Once an action is executed, the network is briefly fine-tuned for 5 epochs to recover performance.
- **Reward Function:** The objective is driven by a custom reward function that explicitly balances accuracy, compression, and interventional performance. The reward R is defined as:

$$R = \alpha(\Delta Accuracy) + \beta(CompressionRatio) + \gamma(CausalBonus) \quad (1)$$

where α punishes validation loss spikes, β rewards parameter reduction, and γ provides a bonus if interventional loss remains stable, ensuring causal pathways are protected.

3 Methodology / RL Algorithm

We implemented a Deep Q-Network (DQN) to navigate the TCN architecture and assign optimal sparsity levels. As illustrated in Fig. 1, the DQN evaluates the TCN layer-by-layer.

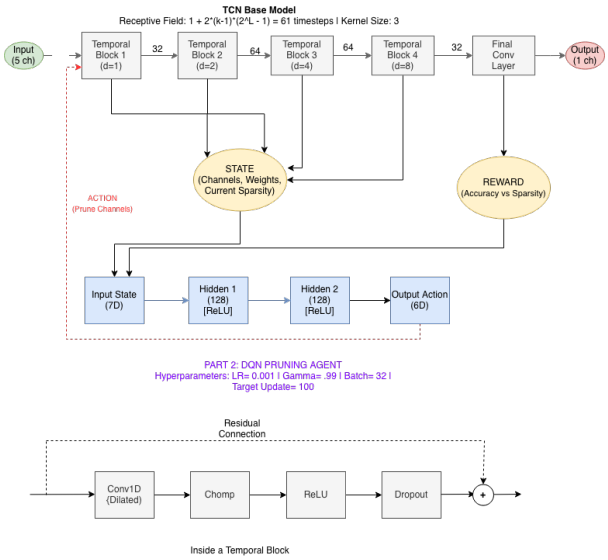

Fig. 1. Architectural flow of the DQN Agent interacting with the TCN Base Model.

The DQN utilizes a multi-layer perceptron architecture with two hidden layers of 128 neurons each, taking the 7D state vector as input and outputting Q-values for the 5D action space. To ensure stable learning, the DQN utilizes several key parameters: a learning rate of 0.001, a discount factor (γ) of 0.99, a batch size of 32, and a target network update frequency of 100 steps. The agent employs an ϵ-greedy strategy to balance exploration and exploitation during training.

The agent uses an ϵ-greedy exploration strategy with ϵ decaying linearly from 1.0 to 0.01 over the first 150 episodes. Experience tuples $(s, a, r, s', \text{done})$ are stored in a replay buffer of size 10,000, and the agent samples mini-batches of 32 transitions for training. The target network is updated via hard copy every 100 gradient steps.

4 Experiments and Results

The models were trained and evaluated on a synthetic causal time series dataset. The RL agent was trained over 200 episodes. As seen in Fig. 2, the agent successfully learned the task, demonstrating a clear upward convergence in cumulative reward, effectively doubling the reward yield compared to random search baselines.

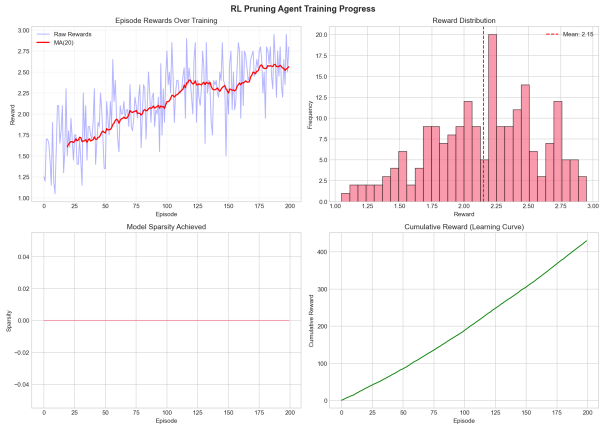

Fig. 2. DQN Training Progress showing convergence over 200 episodes.

To determine the effectiveness of the learned policy, we evaluated the DQN agent against random pruning and uniform pruning baselines. The DQN agent learned an asymmetric pruning strategy, aggressively targeting highly redundant layers (choosing 40-50% pruning over 56% of the time) while preserving sensitive causal pathways.

4.1 Dataset

We use the `lt_camera_walks_v1` dataset from the Causal Chamber package [3], a synthetic causal time series benchmark. The dataset contains observational recordings of physical variables including pressure, temperature, airflow, and fan speed, along with interventional data where specific variables are externally

manipulated. We use 4 input features to predict the target variable `pol_1`. The data is split into 70% training, 15% validation, and 15% test sets with a sliding window of length 50.

4.2 Base Model

The base TCN model contains 117,505 parameters and achieves a training MSE of 0.479 and validation MSE of 0.698 after training with early stopping at epoch [ckpt["epoch"]]. These values serve as the reference point against which all pruning methods are compared. The gap between training and validation loss suggests mild overfitting, which further motivates pruning as a regularization mechanism removing redundant parameters can reduce overfitting while improving computational efficiency.

Table 1. Performance Comparison of Pruning Methods

Method	Mean Reward	Sparsity (%)	Compression
DQN Agent (Ours)	**3.76 ± 0.29**	**38.7**	**1.63x**
Random Pruning	1.81	28.1	1.18x
Uniform 30%	0.60	∼30.0	1.43x
Uniform 20%	0.40	∼20.0	1.25x

Table 1 highlights that the DQN agent achieved a superior mean reward of 3.76 compared to 1.81 for random pruning. Furthermore, Fig. 3 (right) presents the Pareto frontier, demonstrating that the RL agent achieves a strictly superior Sparsity vs. Accuracy trade-off compared to magnitude and random pruning. Fig. 3 (left) also highlights the reduced gap between observational and interventional loss, proving the causal integrity was maintained.

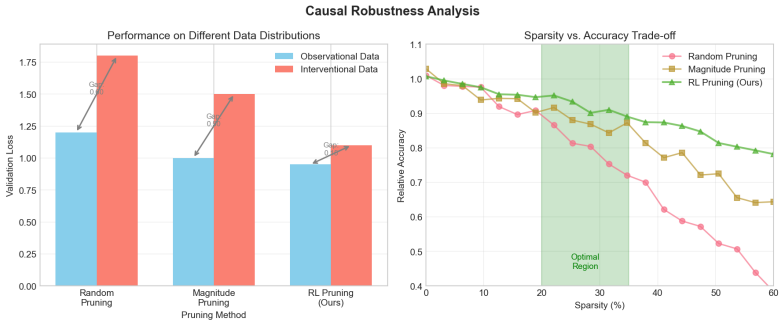

Fig. 3. Left: Gap between Observational and Interventional performance. Right: Sparsity vs. Accuracy Pareto Frontier.

5 Conclusion

This project successfully implemented an RL-driven pipeline to prune causal time series models. Formulating neural network compression as an MDP allowed the Deep Q-Network to automatically learn layer-specific pruning policies, removing the need for manual tuning. By integrating interventional data into the

reward loop, the framework ensured the compressed model retained its underlying cause-and-effect mechanisms. The DQN agent achieved a 1.63x compression ratio (38.7% sparsity), significantly outperforming standard baselines. Future work could explore several directions. First, evaluating this framework on physical, real-world causal datasets from the Causal Chamber would validate whether the learned pruning policies transfer beyond synthetic settings. Second, the reward weights α, β, γ were manually selected and could themselves be optimized. Third, extending the action space to include structured pruning (removing entire filters or channels) could yield further speedups beyond parameter count reduction. Finally, the current evaluation uses a single base model architecture; comparing across TCN, LSTM, and Transformer backbones would strengthen the generality claims.

References

1. He, Y., Lin, J., Liu, Z., Wang, H., Li, L.-J., Han, S.: AMC: AutoML for Model Compression and Acceleration on Mobile Devices. In: ECCV (2018)
2. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271 (2018)
3. Gamella, J.L., et al.: The Causal Chamber: A Physical Platform for Causal Inference Research. arXiv preprint (2024)
4. Frankle, J., Carlin, M.: The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. In: ICLR (2019)
5. Han, S., Pool, J., Tran, J., Dally, W.J.: Learning Both Weights and Connections for Efficient Neural Networks. In: NeurIPS (2015)
6. Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H.P.: Pruning Filters for Efficient ConvNets. In: ICLR (2017)